

Studiare senza riflettere è inutile. Riflettere senza studiare è pericoloso.
~Confucio

DOMANDE APERTE PRIMA PARTE

TEORIA DELL'INFORMAZIONE

- 1 Si descriva il problema dell'overflow.
- 2 Si descrivano le codifiche che conoscete per rappresentare numeri interi positivi e negativi.
- 3 Si abbia un codice di n bit rappresentante un numero x , il cui valore assoluto sia compreso tra 0 e $(2^{(n-1)} - 1)$. Si descriva come ottenere il codice in complemento a 2 rappresentante il numero $-x$.
- 4 Si dica a quale prezzo i numeri in virgola mobile consentono di rappresentare valori in un intervallo molto maggiore di quello consentito da rappresentazioni in virgola fissa.

CIRCUITI LOGICI

- 5 Si descrivano gli ingressi e le uscite di full-adder.
- 6 Si descriva la differenza tra macchine sequenziali e combinatorie.
- 7 Con riferimento al formalismo grafico per rappresentare macchine a stati finiti, cosa viene rappresentato dagli archi del grafo?

ASSEMBLY E CATENA PROGRAMMATIVA

- 8 Si spieghi la funzione del sottoinsieme dei bit di una istruzione macchina chiamato "codice operativo".
- 9 Indicare quali sono le relazioni fra i formati R e J delle istruzioni MIPS ed i seguenti aspetti chiave dell'architettura: numero di registri accessibili al programmatore e dimensioni della finestra di salto raggiungibile con una istruzione di salto incondizionato.
- 10 Si spieghi la differenza tra salti condizionati e incondizionati.
- 11 Si spieghi la differenza, nell'architettura MIPS32, tra i salti incondizionato "j" e quelli "jr".
- 12 Si spieghi la differenza tra salti assoluti e relativi.
- 13 Si spieghi perché, nell'architettura MIPS32, i 16 bit del campo immediato di una istruzione "beq" consentano di identificare una tra 2^{18} locazioni di memoria.

14 Si spieghi perché, nella architettura MIPS32, i 26 bit del campo address di una istruzione "j" consentano di identificare un'altra 2^{28} locazioni di memoria

15 Si spieghi la differenza tra istruzioni macchina e pseudoistruzioni.

16 Nell'architettura MIPS la lunghezza in bit di una istruzione è uguale alla dimensione dei registri general purpose della CPU: questa condizione secondo voi è necessaria? Quali sarebbero vantaggi e svantaggi di modificarla?

17 Si descriva il funzionamento del componente "x" della catena programmatica, chiarendo quali siano gli ingressi di questo componente e quali le uscite prodotte. Si richiede anche di specificare da chi vengano predisposti gli ingressi ed in quale momento questa predisposizione abbia luogo.

x = assemblatore, compilatore, linker, loader, debugger

18 Si elenchino i componenti di una catena programmatica, descrivendo brevemente le funzioni principali che ciascun componente svolge.

DOMANDE CHIUSE PRIMA PARTE

TEORIA DELL'INFORMAZIONE

1 Il numero decimale 23 e' rappresentato dal numero binario:

2 Il numero binario 1101101 e' rappresentato dal numero esadecimale:

3 Si determini il risultato su 8 bit della somma dei seguenti numeri binari in complemento a due: 1111 1100 + 00010001.

4 Se 100 e 110 sono numeri in complemento a due di tre cifre binarie quanto vale $(100 - 110)$?

5 Qual è l'intervallo dei valori rappresentabili in complemento a due su 4 bit?

6 Al massimo quanti caratteri in codifica ASCII possono essere memorizzati in un registro di 32 bit?

CIRCUITI LOGICI

7 Qual è il massimo numero di ingressi che può avere un multiplexer con 4 linee di selezione?

8 Quante linee di selezione sono necessarie in un multiplexer con 6 ingressi?

9 Quante linee di selezione sono necessarie in un decoder con 8 linee d'uscita?

[MACCHINE A STATI FINITI NON PRESENTI]

ASSEMBLY

10 A ciascuna pseudo-istruzione MIPS32 corrisponde:

- una o più istruzioni macchina
- esattamente due istruzioni macchina
- nessuna istruzione macchina
- esattamente una istruzione macchina

11 Le pseudo-istruzioni dell'assembly MIPS32 servono anche a:

- risparmiare tempo di esecuzione
- risparmiare spazio occupato dall'eseguibile in memoria
- migliorare le prestazioni del processore
- risparmiare sul numero dei registri usati
- a migliorare la leggibilità del codice

12 In generale, una pseudo-istruzione

- è sempre più veloce di una istruzione macchina nativa
- richiede sempre più memoria di una istruzione macchina nativa
- impiega le stesse risorse delle istruzioni in cui viene tradotta
- è sempre più lenta di una istruzione macchina nativa
- richiede sempre meno memoria di una istruzione macchina nativa

13 Nell'architettura MIPS32 quanti sono i bit presenti in una parola di memoria?

14 Nell'architettura MIPS32 con quanti bit viene codificata una istruzione?

15 Nell'architettura MIPS32 quanti bit occorrono per specificare un registro generale (specificare = indicare quale registro, tra i registri presenti nella cpu)?

16 Nell'architettura MIPS32 qual è la più piccola quantità di memoria indirizzabile con una singola istruzione macchina (prescindendo dal fatto che il dato sia allineato o meno)?

17 Nell'architettura MIPS32 quanti bit sono presenti in un registro general-purpose della cpu?

18 Nell'architettura MIPS32 quanti registri general-purpose sono presenti in cpu?

19 Nell'ISA (Instruction Set Architecture) del MIPS32 la lunghezza in bit di una istruzione macchina è fissa?

20 Nella sintassi dell'assembly MIPS32, che cosa rappresenta la stringa "\$0"?

- 21** A quale istruzione simbolica MIPS corrisponde il codice macchina esadecimale 0xad0a0000?
- 22** Considerando il seguente spezzone di codice si specifichi qual è il valore (in decimale) specificato nei meno significativi 16 bit dell'istruzione macchina beq \$9, \$8, L1
- 23** A quale codice macchina espresso in esadecimale corrisponde la istruzione simbolica MIPS 'jr \$31'?
- 24** Data l'istruzione macchina il cui simbolico è "addi \$7, \$3, 10", il campo costituito dai bit dal 25 al 21 compresi avranno valore pari a:
- 25** Un'etichetta, in un sorgente assembly, identifica in generale:
- una variabile utilizzata dal programma
 - un registro della macchina
 - il punto di ingresso di una procedura
 - una locazione di memoria
- 26** Nell'assembler MIPS/SPIM le etichette:
- si riferiscono a locazioni di memoria e sono quindi gestite dall'assemblatore
 - sono istruzioni simboliche e sono quindi gestite dall'assemblatore
 - sono pseudo-istruzioni e sono quindi gestite dall'assemblatore
 - sono istruzioni macchina e sono quindi gestite dall'hardware
 - si riferiscono a registri macchina e sono quindi gestite dall'hardware

PROCEDURE

ACQUISIZIONE	CHIAMATA
SALVATAGGIO RISULTATI	CHIAMATA
SALVATAGGIO PARAMETRI	CHIAMANTE
RITORNO	CHIAMATA
TRASFERIMENTO	CHIAMANTE

DOMANDE APERTE SECONDA PARTE

DATAPATH

- 1** Si spieghi perché il circuito "register file" fornisce in uscita il contenuto di due suoi registri contemporaneamente

- 2** Si spieghi la funzione svolta dal registro CPU chiamato Program Counter.
- 3** Si spieghi la funzione svolta dal registro CPU chiamato Instruction Register.
- 4** Quali sono i principali tipi di segnali che si scambiano datapath e unità di controllo e a che cosa servono ?
- 5** Se si volessero implementare 64 registri general purpose in una architettura MIPS, che conseguenze ci sarebbero sul formato delle istruzioni ?

PIPELINING

- 6** Si commenti la correttezza della seguente affermazione: "Il pipelining consente ad un maggiore numero di istruzioni di raggiungere il completamento, per unità di tempo, ma non diminuisce il tempo perché un'istruzione venga completata".
- 7** Si chiarisca se una lunga sequenza di istruzioni, priva di salti condizionati, dia luogo ad una migliore o peggiore performance in una cpu con pipelining.
- 8** Si descrivano degli esempi di "data hazards" e si accenni a come questi si potrebbero affrontare, potendo intervenire sull'hardware della cpu
- 9** Si descrivano i vantaggi e gli svantaggi di avere cache di primo livello separate per istruzioni e dati in una architettura di cpu con pipelining.
- 10** Si descriva cosa si intende per pipeline stall.
- 11** Si commenti la correttezza della seguente affermazione: "Il pipelining consente ad un maggiore numero di istruzioni di raggiungere il completamento, per unità di tempo, ma non diminuisce il tempo perché una istruzione venga completata".
- 12** Si chiarisca se una lunga sequenza di istruzioni, priva di salti condizionati, dia luogo ad una migliore o peggiore performance in una cpu con pipelining

I/O

13 Si discuta la correttezza dell'affermazione "la gestione delle periferiche a controllo di programma consente di gestire periferiche più critiche in termini di latenza e capacità di trasferimento dati (banda passante) di quelle gestibili con la tecnica interruzione di programma".

14 Si spieghi per quali aspetti la tecnica di ingresso/uscita chiamata accesso diretto alla memoria (DMA) è migliorativa rispetto alle tecniche di controllo di programma ed interruzione di programma.

15 Ha senso ipotizzare una architettura di I/O che utilizzi esclusivamente il DMA e non preveda la possibilità di interrupt?

16 Come devono essere gestiti i registri della CPU in caso di interrupt?

ECCEZIONI

17 Descrivere quali operazioni effettua l'hardware nella attivazione di un gestore di eccezioni.

18 Si descrivano i vantaggi e gli svantaggi di avere più gestori delle eccezioni, a indirizzi predeterminati, dedicati a specifiche classi di eccezioni o interrupt.

CACHE

19 Si descrivano i vantaggi e gli svantaggi di avere una cache di primo livello molto grande, ad esempio di alcuni megabyte.

20 Si descrivano gli approcci chiamati write-through e write-back per la gestione delle scritture in memoria, chiarendo i vantaggi che ciascun approccio fornisce.

21 Si motivi la scelta, comune in molti processori, di selezionare il blocco di cache da sostituire in modo casuale

22 Si descriva il funzionamento di una gerarchia di memoria che preveda più livelli di cache.

23 Si descrivano gli approcci chiamati write-through e write-back per la gestione delle scritture in memoria, chiarendo i vantaggi che ciascun approccio fornisce.

24 Si descrivano i vantaggi e gli svantaggi di avere una cache di primo livello molto grande, ad esempio di alcuni megabyte.

DOMANDE CHIUSE SECONDA PARTE

I/O

1 Con riferimento ai principi di funzionamento dell'I/O visti a lezione, rispetto all'I/O a controllo di programma e su Interruzione, la modalità di I/O in DMA è, tra le altre cose:

- un'alternativa del tutto comparabile in termini di efficienza, economia e prestazioni alle altre tecniche di I/O
- più efficiente in termini di uso del processore principale
- più veloce in termini di tempo effettivo impiegato per il trasferimento dati
- più economica in termini di componenti elettronici richiesti

2 Con riferimento ai principi di funzionamento dell'I/O visti a lezione, si indichi un motivo plausibile per cui la modalità di I/O in DMA è più efficiente rispetto all'I/O a controllo di programma e ad Interruzione:

- perché il DMA è migliaia di volte più veloce della CPU nel trasferire i dati tra Periferiche e Memoria
- perché, a differenza della CPU, il DMA non deve effettuare busy wait
- perché il DMA è migliaia di volte più veloce delle altre tecniche nel completare l'I/O
- perché, a differenza della CPU, il DMA effettua il polling della periferica molto più velocemente
- perché con il DMA la CPU è impegnata solo per predisporre l'inizio e gestire il termine dell'I/O

3 Con riferimento ai principi di funzionamento dell'I/O, il DMA richiede una circuiteria apposita, con capacità di accesso al bus e alle periferiche e per questo è tipicamente realizzata da:

- un circuito dedicato all'interno della CPU
- un pezzo di codice situato nel gestore delle Eccezioni di sistema e attivato dalle Interruzioni delle periferiche
- un circuito dedicato all'interno della Memoria principale
- un circuito dedicato, concettualmente pensabile interno alla Periferica

4 Con riferimento ai principi di funzionamento dell'I/O, il processore dopo aver avviato il DMA:

- è libero di proseguire l'esecuzione di programmi
- è costretto a fare polling sulla periferica per verificare se è pronta, e di conseguenza abilitare e disabilitare il DMA
- è costretto a fare busy wait finché i dati non sono stati trasferiti
- è costretto a controllare il DMA finché il trasferimento non è terminato

5 Con riferimento ai principi di funzionamento dell'I/O, il DMA richiede l'utilizzo di:

- piu' tempo di CPU per poter completare il trasferimento dei dati
- un set di istruzioni apposite per essere eseguito
- meno registri periferica rispetto al controllo di programma
- piu' registri periferica rispetto al controllo di programma

6 Con riferimento ai principi di funzionamento dell'I/O, nel caso di I/O Memory Mapped il bit di Ready di una periferica, fisicamente e' realizzato:

- nel registro Status del coprocessore 0
- nel registro Cause del coprocessore 0
- nei circuiti di una periferica di I/O, ma appare come uno dei bit di una specifica locazione dello spazio di indirizzamento di memoria
- da una word di memoria RAM del sistema e quindi appare come uno dei bit di una normale locazione di memoria
- in un registro generale del processore

7 Con riferimento ai principi di funzionamento dell'I/O, nel caso di I/O Memory Mapped, la CPU puo' fare I/O:

- solo con istruzioni del processore riservate all'I/O
- solo nel gestore eccezioni
- solo con il DMA
- accedendo ai registri dati e controllo della periferica come se fossero normali locazioni di memoria

8 Con riferimento ai principi di funzionamento dell'I/O, qual e' il vantaggio principale dell'I/O Memory Mapped?

- Il set di istruzioni della CPU non deve essere esteso con istruzioni specifiche per accedere ai dispositivi di I/O
- Semplifica la circuiteria delle periferiche, che quindi sono piu' economiche
- Permette l'I/O su Interruzione, altrimenti impossibile
- E' piu' efficiente in termini di velocita' di trasferimento dati con le periferiche

ECCEZIONI

9 Quali registri generali deve preservare un gestore delle eccezioni MIPS?

- tutti i registri generali utilizzati, compreso \$at, ma esclusi \$k1 e \$k0
- solo i registri \$k1 e \$k0
- solo i registri del Coprocessore 0
- i registri generali utilizzati e tutti i registri del Coprocessore 0

10 Se il registro Cause assume il valore x, quale eccezione si e' verificata?

x=0x00000100, 0x00000018, 0x00000200, 0x00000000, 0x00000020, 0x00000014, 0x00000030

11 Nell'architettura di riferimento MIPS, secondo le convenzioni software utilizzate da PC SPIM, il gestore delle eccezioni puo' essere collocato:

- solo a partire dalla locazione 0x8000180

- ovunque nel segmento text
- ovunque nel segmento data
- ovunque in memoria

CACHE

12 Un segnale periodico ha periodo $5 \cdot 10^{-5}$ s. Qual e' la frequenza ad esso associata?

13 Un segnale periodico ha frequenza di 33MHz. Qual e' il suo periodo?

14 Considerando di avere una cache di x blocchi di y words, a che numero di blocco viene mappato l'indirizzo z(indirizzamento al byte) ?

x= 16, 16, 4, 8, 8, 8
 y= 4, 8, 8, 16, 4, 8
 z= 64, 128, 128, 256, 64, 128

15 Data una macchina con 10 CPI per accessi solo in cache, 15 cicli di miss penalty (intesa come tempo di trasferimento blocco + accesso) e hit probability 0,8, qual e' il numero medio di CPI ?

16 Data una macchina con 2.2 GHz di frequenza di clock, 10 CPI per accessi solo in cache, 15 cicli di miss penalty(intesa come tempo di trasferimento blocco + accesso) e hit probability 0,8, qual e' velocità media in istruzioni/sec?

17 Si supponga di avere una cache a mappaggio diretto con 4 blocchi da 2 word. Se la cache parte vuota e si esegue 2 volte un loop che utilizza le locazioni di memoria 5, 2, 3, 4, 4, quanti miss ci saranno?

18 Si supponga di avere una cache a mappaggio diretto con 4 blocchi da 2 word. Se la cache parte vuota e si esegue 6 volte un loop che utilizza le locazioni di memoria 7, 5, 6, 4, 7 (indirizzamento alla word, non al byte), quanti miss ci saranno ?

19 Si supponga di scrivere dati in maniera sequenziale in un vettore posizionato in memoria da 0x00100000 a 0x0010007FF (estremi inclusi, indirizzamento al byte). Vi sia una cache a mappaggio diretto con 16 blocchi da 4 word; se la cache, inizialmente vuota, alloca sulla scrittura e adotta una politica di write-back, quanti write-back ci saranno dopo aver scritto tutto il vettore?

DATAPATH

20 Facendo riferimento alla CPU multiciclo vista a lezione, riportata per comodita' in figura, e alla relativa logica di controllo per la implementazione delle istruzioni add, and, beq, j, lw, or, slt, sub, sw, si chiede la lista di tutte le istruzioni in cui x

avviene una scrittura nel register file:

- tutte le istruzioni R-type (add, and, or, slt, sub)
- add, and, lw, or, slt, sub

- add, lw, or, sub, sw
- tutte meno beq e j

la ALU usa il registro A come sorgente di una delle sue operazioni:

- tutte meno j
- tutte meno beq e j
- tutte le istruzioni aritmetico-logiche (add, and, or, sub)
- add, and, beq, lw, or, slt, sub

la ALU usa il registro B come sorgente di una delle sue operazioni

- tutte meno beq e j
- add, and, beq, or, slt, sub
- tutte le istruzioni R-type (add, and, or, slt, sub)
- add, and, or, sub

viene utilizzato il contenuto del registro B

- add, and, beq, or, slt, sub, sw
- tutte le istruzioni R-type (add, and, or, slt, sub)
- tutte meno j, lw e sw
- add, and, or, sub

durante la cui realizzazione (fetch + execute) la ALU effettua tre somme.

- in nessuna, la ALU non ha pipeline ed effettua una operazione aritmetica per istruzione
- add, beq, j, lw, sw
- add
- add, lw, sw

durante la cui realizzazione (fetch + execute) la ALU effettua due o più somme.

- add, lw, sw
- tutte le istruzioni
- add, beq, lw, sw
- in nessuna, la ALU non ha pipeline ed effettua una operazione aritmetica per istruzione

durante la cui realizzazione (fetch + execute) la ALU effettua due somme ed una sottrazione.

- sub, beq, slt
- beq, slt
- sub, beq
- in nessuna, la ALU non ha pipeline ed effettua una operazione aritmetica per istruzione

durante la cui realizzazione (fetch + execute) la ALU effettua esattamente due somme (e non tre).

- add, beq, j, lw, sw
- and, beq, j, or, slt, sub
- in nessuna, la ALU non ha pipeline ed effettua una operazione aritmetica per istruzione
- add, lw, sw

durante la cui realizzazione (fetch + execute) la ALU effettua una sottrazione.

- sub
- sub, beq

- beq, slt, sub
- sub, slt

21 Facendo riferimento alla CPU multiciclo vista a lezione, riportata per comodita' in figura, e alla relativa logica di controllo, si supponga di affrontare in modo diverso da quanto visto a lezione il problema di salvare il PC in EPC per la gestione delle eccezioni. Si supponga di salvare all'inizio di ogni istruzione (fase di fetch) il valore del PC in EPC, utilizzando hardware apposito, in modo da evitare la sottrazione di 4 necessaria se il salvataggio avviene nelle fasi successive. Tale soluzione funziona ?

- funziona, il PC viene salvato, ma non si risparmia nulla in termini di cicli di clock